

VCF Core: A Semantic Foundation for RDF Representations of VCF

Elias Crum^{1,2,*}, Bart Buelens², Gökhan Ertaylan² and Ruben Taelman¹

¹IDLab, Department of Electronics and Information Systems, Ghent University – imec, Belgium

²Flemish institute for Technological Research (VITO) Mol, Belgium

Abstract

Medical applications of genomic data depend on connecting variant observations to clinical information and biomedical knowledge. Linked Data offers a framework for this integration, but the Variant Call Format (VCF) does not natively provide an RDF representation of the observations and their interpretation context. We present VCF Core Vocabulary 2.1.2, a shared semantic target for representing VCF files, header declarations, records, annotations and sample data. The vocabulary integrates existing provenance and genomic models while retaining VCF-specific field definitions, ordering and missingness. Expanded sample calls expose individual values, whereas condensed, sample-ordered vectors provide an alternative for cohort data. Version-specific registries and validation profiles address VCF 4.1–4.5, including 122 reserved INFO and FORMAT declarations for VCF 4.5. A curated assessment records representation evidence for all 104 inventoried logical-model constructs and enforcement evidence for 87, and a set of 123 curated alignments documents connections to existing variation models and reference ontologies. We evaluate the vocabulary with executed checks rather than term counts. A specification-derived assessment of 94 requirements over VCF 4.1–4.5 supplies queries and expected answers authored in advance; replaying the 39 requirements that a ten-fixture subset exercises over graphs from a second, separately implemented producer gives agreement on 536 of 572 checks, 508 of them passing for both, with no divergence attributable to a representation gap. A semantic round-trip recovers the fixed columns and genotypes of all 35 records, in the expanded profile, from structured properties alone, and one integration example answers an allele-dependent question under VCF 4.5 local alleles with checked answers and recovered provenance. The replay also found a silent correctness defect in local-allele handling that per-producer testing had not, which we report as evidence for keeping representational mechanism, implementation and correctness apart. We then discuss how this foundation could support reusable conversion engines, traceable links to biomedical resources, and SPARQL queries combining genomic observations with medically relevant information.

Keywords

Variant Call Format, RDF, Genomic data, Vocabulary, Data integration, SHACL

1. Introduction

Genomic data are increasingly useful in medicine when integrated with other patient, public and proprietary information. A drug-response assessment, for example, depends on the relationship between a patient’s genomic findings, a prescribed medication, and the knowledge used to interpret that relationship [1]. The presence of a variant alone does not establish these connections. Semantic approaches provide a basis for making such connections explicit: HERO-Genomics addresses integration of genomic and biomedical information, while Semantic Beacons demonstrates federated queries over variant data and public knowledge graphs [2, 3]. Maturing investigation of genomic–medical data integration motivates a reusable representation of the source observations on which interpretation depends.

The Variant Call Format (VCF) is widely used to exchange genomic variant data [4]. It already supports database identifiers, reference information and extensible annotations, so its limitation is not that data linking is impossible. Rather, VCF does not natively expose these elements and their

SWAT4HCLS 2027, February 1–4, 2027, Basel, Switzerland

*Corresponding author.

✉ elias.crum@ugent.be (E. Crum); bart.buelens@vito.be (B. Buelens); gokhan.ertaylan@vito.be (G. Ertaylan); ruben.taelman@ugent.be (R. Taelman)

ORCID 0009-0005-3991-754X (E. Crum); 0000-0001-7734-3747 (B. Buelens); 0000-0001-5602-6435 (G. Ertaylan); 0000-0001-5118-256X (R. Taelman)



© 2027 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

relationships in a Findable, Accessible, Interoperable and Reusable (FAIR) way. Moreover, a value in a VCF file often depends on a header declaration, an ordered allele list, or the structure of a sample column for its interpretation [5]. Converting only selected variant attributes can leave those dependencies implicit. Linked Data and the Resource Description Framework (RDF) are therefore an attractive way to capture the full context of the information a VCF file carries.

Existing VCF-to-RDF approaches provide important foundations. The Genomic Feature and Variation Ontology (GFVO) includes VCF term coverage within a broader model of genomic features and variation, and VCF2RDF presents an isomorphic semantic mapping of the VCF format [6, 7]. The next step is therefore not to establish that VCF can be represented in RDF. It is to provide a shared, version-aware target that combines broad coverage of the source format with explicit connections to these existing models. Such a target would allow conversion and querying approaches to develop without each introducing its own interpretation of VCF structure.

Here, we present the VCF Core Vocabulary (*vcfcv*) 2.1.2 [8], developed from our earlier VCF-RDFizer Vocabulary [9]. The contribution combines source-context preservation, alternative sample representations, documented alignment, and coverage evidence across VCF 4.1–4.5. After introducing VCF and the model, we explain its relationship to existing work, assess its scope, and discuss how it could support integrated, medically relevant genomic querying. Throughout, *vcfc* : denotes <https://w3id.org/vcf-core/vocab#>.

2. VCF as a semantic representation problem

2.1. What a VCF file describes

A VCF file contains meta-information lines, a column-header line and data records. Meta-information lines beginning with `##` identify the format version and can describe the reference sequence, genomic sequence segments called contigs, and the fields used in the records. The column header begins with `#CHROM`. Each subsequent record describes a genomic position, the reference sequence at that position and one or more alternatives, together with observations and annotations [5]. Table 1 summarizes the main components.

Table 1

VCF components and the interpretation context that an RDF representation needs to retain. INFO describes a record; FORMAT describes the contents of sample columns.

Component	Role in the source file
Header declarations	Define field identifiers, types, cardinalities and descriptions, together with reference and processing context.
CHROM, POS, ID	Identify the reference contig, position and optional record identifiers. Position alone is not a globally unique variant identifier.
REF, ALT	Give the reference allele and ordered alternative alleles. An allele is a sequence alternative at the represented locus.
QUAL, FILTER, INFO	Describe call quality, filter status and record-level annotations, including producer-defined fields.
FORMAT and samples	Declare an ordered set of keys and give corresponding values for each sample, such as genotype (GT) and read depth (DP).

For example, a synthetic record at position 100 on contig 1 with reference allele A, alternative allele G and `FORMAT=GT:DP` may carry the sample values `0/1:42`, `0/0:18` and `./.:.` — a heterozygous call at depth 42, a homozygous reference call at depth 18, and a sample whose genotype and depth are both missing. The slash separates unphased allele calls, while a vertical bar marks phased calls in which allele ordering is meaningful [5].

These compact encodings illustrate why representing VCF requires more than assigning predicates to columns. Genotype indices refer to the record’s ordered allele list, and the meaning of a sample value

depends on its FORMAT key. Likewise, a header's `Number=A` means one value per alternative allele, whereas `Number=R` includes the reference allele and `Number=G` depends on possible genotypes. The same key, such as `DP`, can occur in `INFO` and `FORMAT` without identifying the same observation [5].

2.2. Why version and source context matter

VCF 4.1–4.5 share these basic structures but differ in supported declarations and constraints [10, 5]. For example, CIPOS describes uncertainty around a reported position. Its declared cardinality changes from two values in VCF 4.1–4.3 to two per alternative allele in 4.4–4.5. VCF 4.5 also introduces local-allele cardinalities and base-modification fields. A common RDF model should support these developments without interpreting an older file under rules introduced later.

A source observation must also remain distinguishable from its biological interpretation. Two files may describe the same alteration with different quality scores or annotations. A sample column may represent a patient specimen, a pooled sample or a synthetic example. Neither records nor sample columns should therefore automatically be equated with normalized variants or patients.

3. The VCF Core vocabulary

3.1. Retaining observations and their provenance

VCF Core separates a `VCFFile`, its `VCFHeader`, individual `VCFRecord` resources and a `VariantCall` associated with each record. The call groups quality, filter, annotation and genotype information; it is a modeling abstraction rather than an additional line in the source file. Figure 1 shows this shared structure and the two sample representations.

The file specializes `dc:Distribution` and `prov:Entity`, while records and calls provide further attachment points for provenance [11, 8]. File-based IRI templates preserve the relationship between source, record and field. Because a VCF identifier can be missing or non-unique, a producer must use a record identifier that distinguishes rows within the source. This approach allows observations from different files to remain separate even after they have been linked to a common biological entity.

The property `asSequenceAlteration` connects a record or call to a separately described Sequence Ontology (SO) sequence alteration [12]. FALDO supplies location, reference and position descriptions, including uncertain positions where appropriate [13]. Thus, VCF Core does not introduce a competing general-purpose location model. Establishing cross-file links still requires compatible reference sequences and correct allele interpretation; the vocabulary provides the connection points rather than a normalization algorithm.

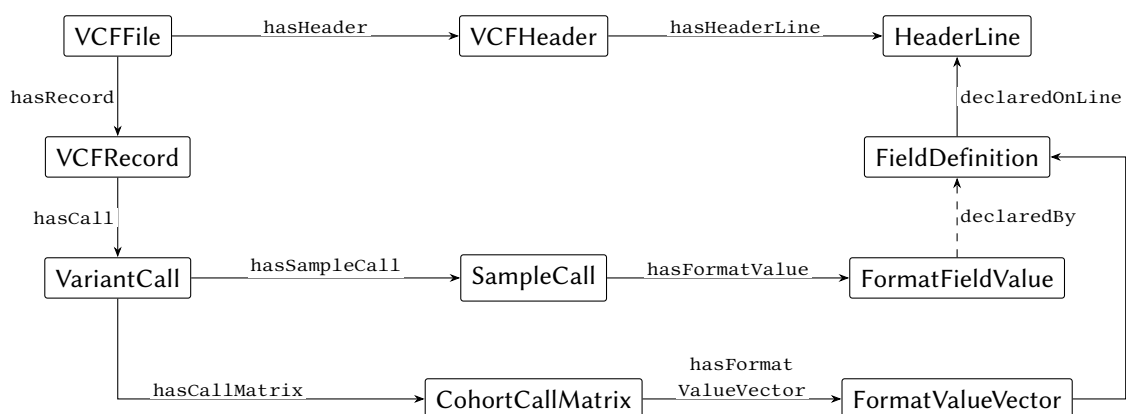


Figure 1: Selected VCF Core relationships; prefixes are omitted. Both sample profiles retain the file, record and header context. An expanded value can link to its definition through `declaredBy`; the equivalent relation is required for a condensed vector. Sample identity and ordering are described separately through `VCFSample` and `SampleSet`.

3.2. Making field interpretation explicit

Header declarations are represented because they determine how values should be read: VCF Core gives each meta-information line type its own class and exposes a field definition's identifier, Number, Type, Description and optional source and version as data, with generic `HeaderAttribute` resources retaining producer-defined additions [8].

Two modelling decisions in this area carry most of the weight. First, a standardized definition and its occurrence in a file are kept apart — the reserved `FORMAT/DP` definition describes a key the specification supplies, while a header line records what one file declares — so `FieldDefinition` and `HeaderLine` are distinct resources joined by `declaredOnLine`, and values reach their declaration through `declaredBy`. Without that separation a registry entry would be forged into every file that never declared it. Second, lexical and parsed forms are both retained: `fieldValue` keeps the source token or list while optional integer, decimal and boolean properties serve scalar queries, so a missing depth is never coerced to zero and a non-finite float is never forced into `xsd:decimal`. Standalone missing values use `vcfc:Null`, and ordered allele resources index `REF` at zero and `ALT` upwards, which is what makes a genotype integer and an allele-dependent annotation resolvable. Raw source text may be retained alongside, but a preserved string is never counted as structured coverage.

3.3. Choosing how sample data are represented

The expanded profile connects each call to per-sample `SampleCall` resources and individual `FormatFieldValue` resources. This makes values directly accessible through graph patterns and allows an application to annotate a particular sample observation. A `VCFSample` identifies the source column; linking it to a biological specimen or patient is a separate integration task. Expanded representation is applicable to both single- and multi-sample files when direct access justifies the additional graph structure.

Condensed representation associates each call with a cohort matrix. The `CohortCallMatrix` groups one `FormatValueVector` for each represented `FORMAT` key. A shared `SampleSet` identifies samples by name and one-based index. Each vector specifies its definition, encoding and payload. The initial `VCFTextVector` encoding stores tab-separated sample values in index order, including missing values. The synthetic example therefore produces `GT` values `0/1`, `0/0`, `./.` and `DP` values `42`, `18`, `.` in corresponding positions.

This is a change in graph granularity, not a replacement of sample observations by cohort summaries. Individual condensed cells require decoding before per-sample comparisons; they are not directly bound by ordinary basic graph patterns. For R records, S samples and F `FORMAT` keys, expanded materialization creates RS sample calls and RSF value resources. Condensed materialization creates S sample resources, one sample set, R matrices and RF vectors. These counts exclude shared metadata and do not establish savings in bytes, memory or query time. The underlying payload still contains RSF values.

Listing 1 illustrates direct expanded access. On the supplied synthetic graph it returns `SAMPLE1` with depth `42`; decoding the condensed graph gives the same selection and recovers the same six `FORMAT` cells. The threshold is illustrative, not a clinical quality standard. This bounded example establishes neither general conversion fidelity nor clinical validity.

4. Building on existing semantic models

4.1. Complementary contributions

The usefulness of VCF Core depends on its relationship to the models already used for genomic data. `GFVO` was developed from several genomic formats, explicitly including VCF 4.1 and 4.2 [6]. `VCF2RDF` demonstrated source-preserving semantic conversion [7]. `HERO-Genomics` includes VCF files and records within a broader integration model, while the `Genome Variation Ontology (GVO)`

Listing 1: An expanded-profile query over the supplied synthetic example.

```
PREFIX vcf: <https://w3id.org/vcf-core/vocab#>
SELECT ?sampleName ?depth WHERE {
  ?file vcf:representationProfile vcf:ExpandedRepresentation ;
        vcf:hasRecord/vcf:hasCall/vcf:hasSampleCall ?call .
  ?call vcf:forSample/vcf:sampleName ?sampleName ;
        vcf:hasFormatValue ?value .
  ?value vcf:declaredBy/vcf:fieldId "DP" ;
        vcf:fieldValueInteger ?depth .
  FILTER (?depth >= 20)
}
```

addresses detailed variation descriptions, including complex structural variants [2, 14]. The GA4GH Variation Representation Specification (VRS) provides computable variation descriptions and federated identification, rather than a representation of every source-file detail [15]. These are complementary purposes, not successive attempts at the same task.

To place VCF Core among them without asserting superiority, we inspected the published artifact of each model and scored nine retrieval criteria drawn from the interpretation context a VCF file carries about itself (Table 2). Each criterion is a question about what can be retrieved, not a judgement of quality, and each verdict names the term or absence it rests on. GA4GH VRS is deliberately not scored: it describes computable variation identity rather than a source file, so scoring it against file-format criteria would manufacture a contrast its authors never sought.

Table 2

What each inspected artifact lets a consumer retrieve. **E** explicitly represented; **C** representable with additional conventions; **S** outside the model’s stated scope, which is not a defect. Scored against GFVO (BioInterchange master), HERO-Genomics 1.0, GVO (versionInfo 2021-11-18) and the VCF2RDF term set (v4_2), GFVO and HERO-Genomics retrieved 2026-09-12, GVO and the VCF2RDF term set 2026-09-13.

Retrieval criterion	VCF Core	GFVO	HERO	GVO	VCF2RDF
A record traces to its source file	E	E	E	S	C
A declaration exposes ID, Number, Type	E	S	S	S	E
The file’s VCF version is readable	E	S	S	S	C
ALT allele <i>n</i> is addressable by index	E	C	C	S	C
Ploidy, calls and phasing are separable	E	C	C	S	C
A (sample, field) value without splitting a cell	E	C	C	S	C
A Number=A/R/G item binds to its allele	E	C	C	S	C
An explicit . differs from “not stated”	E	S	C	C	C
The sample encoding is readable from the graph	E	S	S	S	S

Two contrasts explain the pattern. HERO-Genomics and GVO both retain the INFO column as a single literal — our alignment set records `hero:vcfInfo` and `gvo:info` as exact matches to `vcf:infoRaw` — so every INFO-derived question requires re-parsing that string and supplying the cardinality rule from outside the graph. GFVO makes the opposite choice, modelling particular field *meanings* as first-class concepts (AlleleCount, Coverage, MappingQuality), which answers those fields well and leaves the declaration mechanism unaddressed. VCF2RDF is the instructive case: its published term set does define Number, Type and Description as dereferenceable terms, so the cardinality rule is named rather than left outside the vocabulary — but with no allele-index term a consumer must still split the list and count ALT alleles by re-reading a literal. Having the declaration is necessary and, alone, insufficient; binding a value to an allele is the part that must be represented rather than inferred.

These verdicts are bounded in three ways. The criteria were chosen for the interpretation problem VCF Core addresses, so this is not a neutral ranking — a matrix chosen by GVO’s authors would look different and would be equally legitimate. GVO’s column is S in every row but missingness, because the retrieved ontology declares 48 classes, 12 datatype properties and *no* object properties: a model of variation types, doing something other than file modelling. Missingness scores C there only because a literal-valued property can carry a dot, while distinguishing that from an absent triple remains a convention the ontology does not supply. The VCF2RDF column rests on a published term set carrying specification glosses rather than RDF axioms, and on no converter output, so what its graphs contain is still taken from its paper. Finally, “requires additional conventions” states what an inspected artifact establishes, never that a model’s community could not represent something.

4.2. Reuse and curated alignment

Direct reuse and cross-model alignment are kept distinct. PROV, DCAT, SO and FALDO terms contribute to the core representation through axioms or instance descriptions. The accompanying alignment set documents correspondences to models whose assumptions do not always coincide with a source-oriented VCF representation. Table 3 summarizes the 123 mappings from 65 VCF Core terms to 103 external terms [8].

Table 3

Curated alignment targets. Counts describe the mapping set, not comparative specification coverage or demonstrated instance-level interoperability.

Target	Mappings	Connection to VCF Core
GVO	39	Variation and VCF-related properties.
HERO-Genomics	17	File, record and observation descriptions.
GA4GH VRS 2.0	15	Computable descriptions of variation.
GFVO	14	Genomic features and field-related concepts.
Med2RDF	3	VCF-oriented allele properties.
SO, ChEBI, GENO, FALDO	35	Alteration types, modifications, genotype concepts and locations.

Mappings are curated in the Simple Standard for Sharing Ontological Mappings (SSSOM), which records the mapping predicate, justification and associated metadata [16]. A generated Turtle bridge provides the RDF form. Of the 123 mappings, 48 are also asserted inline in ontology modules; the remaining 75 are available through the separately loaded bridge. Keeping this distinction visible lets consumers identify which mapping assertions accompany the core and which require an additional module.

The set contains 57 `skos:exactMatch`, 40 `skos:closeMatch`, 16 `skos:relatedMatch` and 10 `skos:narrowMatch` assertions. These do not establish OWL class or property equivalence, nor do they generate the instance links needed for a biomedical query. They nevertheless have defined semantics: `skos:exactMatch` is transitive, whereas `skos:closeMatch` is not [17]. Consequently, mixed mapping paths cannot be treated as unrestricted rules for predicate substitution, and mappings between different VRS releases require explicit review.

The repository additionally preserves 116 SSSOM mappings from work in the 2025 SWAT4HCLS BioHackathon separately from VCF Core’s own assertions [18]. Of 47 terms previously recorded as lacking a VRS counterpart, eight have a source-oriented VCF Core counterpart, including file, quality, and as-written allele concepts. This illustrates the contribution of retaining source semantics alongside biological models; it does not mean that the 39 remaining annotation and interpretation concepts should all become VCF Core terms.

5. Specification coverage and implementation evidence

5.1. One representation with version-specific rules

VCF Core uses one shared model for common structures and version-specific registries and validation overlays for VCF 4.1–4.5 [8]. The registries contain 40, 40, 67, 78 and 122 explicit reserved declarations, respectively. For VCF 4.5 these comprise 51 INFO and 71 FORMAT definitions, including three entries for parameterized base-modification key families. Prose-only fields in older versions are not assigned Number or Type constraints absent from their specifications.

The overlays condition rules on the owning file’s declared version. Thus, the CIPOS example in Section 2 can be assessed under the appropriate historical or current rule, while the LA, LR, LG and M cardinalities are restricted to VCF 4.5. Shared validation is divided into portable SHACL structure checks, SPARQL-based cross-resource checks and raw/parsed-value consistency checks [19, 8]. This organization permits different validator capabilities without implying that each layer independently validates the entire format.

5.2. What the evidence establishes, and at what strength

Coverage claims are easy to overstate because three different things are routinely reported as one number. We keep them apart. *A mechanism exists* when a term, rule or documented pattern is available; this establishes nothing about correct instantiation. *An example instantiates it* when a fixture demonstrates the mechanism in use; this establishes nothing about general correctness. *Behaviour is independently checked* when an executed query agrees with an expected answer established separately from the code that produced the graph. Only the third supports a correctness claim, and only within what was tested.

Two assessments and two cross-checks address three questions: whether VCF Core preserves the intended meaning of the constructs it claims to support (RQ1); whether the version-specific artifacts and the two sample profiles behave as documented (RQ2); and whether the result supports a reproducible integration task that retains source context (RQ3).

Mechanism. The curated VCF 4.5 inventory enumerates 104 logical constructs across file and header organization, declarations, types and cardinalities, lexical rules, fixed fields and alleles, genotypes, structural variation, repeats and gVCF. All 104 have preserved, structured representations and 87 additionally have an enforcement rule. Separately, all 333 reserved Number/Type rows extracted from the specification texts match the versioned registries [8]. Those 333 rows — 31, 31, 69, 79 and 123 per version — are the keys each source declares explicitly in ##INFO=##FORMAT= blocks or reserved-key tables, and are a different population from the registry entries counted above. VCF 4.1 and 4.2 each define a further 29 reserved keys in prose bullets that give no Number and, for INFO, no Type; those rows are inventoried but cannot be compared, so this agreement figure is weighted towards VCF 4.3–4.5. The inventory was curated by the vocabulary’s developers, so it maps what its authors listed and cannot reveal an omission nobody listed.

Instantiation and independent checking. The specification-derived assessment reads information requirements from pinned VCF 4.1–4.5 sources and tests them against fixtures. It comprises 94 requirements over 210 cases; each case pairs a SPARQL query with an expected answer authored from the specification text *before* the query was run, and every passing query is re-run against an emptied graph as a negative probe. Executing every case on both sample profiles yields 840 nominal query runs, but 165 cases score the identical query and expected answer on two axes, so the independent figure is **465 distinct executions**. Coverage is reported per version because requirements apply per version, and an untested requirement scores zero rather than being excluded (Table 4).

A previous version of this assessment reported 491 reviewed requirements of which 133 had complete evidence. That generation has been superseded: requirements are now scored per version and per profile, and no single figure corresponds to the earlier 133.

Table 4

Reported evidence, its denominator and the claim it supports. The denominators count different things and must not be combined into a single percentage.

Assessment	Result	Supported interpretation
Curated VCF 4.5 inventory	104/104	All inventoried logical constructs have preserved, structured representations.
Enforcement within that inventory	87/104	A stated requirement is checked for these constructs.
Reserved Number/Type rows	333/333	Extracted declarations match the version-specific artifacts.
Requirements with a passing test, by version	32/69, 34/72, 35/74, 38/83, 51/91	Conservative demonstrated coverage for VCF 4.1–4.5; untested requirements score zero.
Distinct executed checks	465	Query runs that are not a repeat of an identical check.
Cross-producer replay	508/572	Checks on which the vocabulary’s materializer and the cited converter agree and both pass.
Semantic round-trip	35/35 records	Fixed columns and genotypes recovered from structured properties alone.
Recorded full validation run	21 fixtures	These examples pass the implemented validation layers.

5.3. Whether the claimed coverage survives a second producer

The assessment above builds its witness graphs with the vocabulary repository’s own materializer, whereas the implementation this paper cites is a separate program, and nothing had established that a query and expected answer written against one also holds over the other — the gap between “the vocabulary can express this” and “the software people will use does”. The two programs share an author but no code: one materializes graphs from a Python fixture builder, the other from RML mappings and streaming emitters. Independence here therefore means independent implementations, not independent authorship, and the check is correspondingly weaker than third-party reproduction.

We replayed the existing queries, against the existing reviewed expected answers, over graphs produced by VCF-RDFizer: ten fixtures spanning VCF 4.1–4.5 and the difficult decisions above, each converted in both profiles, giving 572 checks over 143 cases and 39 requirements. The producers agree on 536 checks and both pass on 508; 34 requirements agree on every check and 28 of those pass throughout. Every check they agree on by both failing is a condensed-profile structure query, the documented storage trade-off.

The exercise earned its keep by failing. Its first run diverged on 15 requirements, nine of which were converter defects precise enough to fix. One was a silent correctness defect in VCF 4.5 local alleles, this paper’s own headline feature: Number=LA and Number=LR items were bound to allele indices positionally, ignoring the sample’s LAA subset, so a depth list of 20, 30, 10 under LAA=2, 4 landed on alleles 0, 1 and 2 instead of 0, 2 and 4 — plausible, well-formed values on the wrong alleles, which no existing test caught because every existing test ran against the other producer. Five requirements still diverge: a QUAL datatype the vocabulary deliberately leaves open, three structured accessors the converter has not built (one of them the condensed-vector decoding path), and one empty-list encoding convention. None is a case of the vocabulary being unable to express something — across all 572 checks, zero divergences trace to a representation gap.

A complementary check recovers logical content rather than comparing query answers. Across the same ten fixtures we reconstruct CHROM, POS, ID, REF, ALT, QUAL, FILTER and the GT subfield from the RDF alone: REF and ALT from ordered `alleleIndex/alleleValue` resources, GT from ordered `GenotypeAlleleCall` resources with their phasing status, so no untouched source string serves as a shortcut. All 35 records are recovered, with three bounds, all by design: INFO and the non-GT FORMAT subfields are out of scope; the check runs on expanded-profile graphs only, since decoding a condensed vector needs the accessor requirement R21 identifies as missing; and in the local-allele fixture, where each site is written once locally and once globally, matching on position confirms one recovery per site

rather than both. It is not whole-file reconstruction.

Where the two sample profiles agree. Because every case runs in both profiles, the profile claim can be stated precisely rather than asserted. The profiles return the same outcome on 252 of 286 comparable checks, and the 34 disagreements fall on nine requirements, every one reading a FORMAT-scoped value: genotype structure, trailing-field missingness, sample-local allele and base-modification items, and repeat components. Record-level file, header, declaration and allele questions agree on every check in both profiles. The supported claim is therefore that the profiles are equivalent for file, header, record and record-level allele questions and divergent by design wherever a query walks per-sample resources. Two of the nine are local-allele requirements, so this paper’s headline feature is among those a condensed graph answers less completely.

Answers to the three questions. RQ1 is answered for the 53 of 94 requirements with at least one executed case, the great majority passing. RQ2 is answered for 17 of the 36 version-conditioned requirements, with the profiles behaving as documented: equivalent above the sample level, divergent below it. RQ3 is answered for 27 of the 38 requirements on the provenance chain an integration query must traverse, which Section 5.5 then exercises end to end. None of these fractions is a conformance claim; each is the share of requirements whose behaviour has been checked at all.

Remaining gaps, classified. Of the requirements exercised, none reveals a representation gap. One correctness defect was found and fixed; three requirements await structured accessors in the converter; one reflects an interpretation the vocabulary deliberately permits both ways; one is an encoding convention. The largest limitation is absence of evidence rather than negative evidence: 41 of the 94 requirements have no test case yet, concentrated in structural-variant fields with version-specific semantics. Allele-dependent values, this paper’s central modelling claim, are tested for 4 of the 11 requirements that bear on them — adequate to demonstrate the mechanism and to catch the defect above, not adequate to claim general correctness. Outside that classification, four validator findings remain from the release’s own conformance checks: custom structured-header IDs can be absent or duplicated without rejection; numeric base-modification keys can be accepted with incorrect Number/Type pairs; a partially missing phase-set-list example is rejected under an unresolved interpretation; and mixed LF/CRLF line endings are rejected under an unconfirmed reading of the specification. Byte-level checking is reported separately, and neither BCF binary layout nor byte-for-byte reconstruction is a target of the logical model.

5.4. Implementation and reproducibility

The release provides OWL/Turtle modules, SHACL profiles, examples, versioned registries, SSSOM mappings and documentation [8]. VCF-RDFizer supplies a separate implementation: release 3.0.3 constructs file and record context with RML mappings and produces expanded or condensed sample structures through streaming emitters [20]. The Section 5.2 assessment was produced with VCF Core 2.1.2 and the vocabulary’s own materializer, with no converter involvement; all converter-dependent results (Sections 5.3–5.5) used VCF-RDFizer 3.0.3, the release that carries the corrections the cross-producer check prompted. Both are tagged, so every figure below comes from a released artifact rather than a working branch. Converting the Section 3.3 example emits 242 expanded and 168 condensed triples, drawing on 93 and 80 distinct VCF Core terms respectively. Every one of those terms is declared in VCF Core 2.1.2, and both graphs conform, with no violations, to the shared and the VCF 4.5 SHACL profile when the shapes are loaded with the vocabulary and RDFS inference, as the vocabulary’s own validation does. The depth query in Listing 1 returns its expected single row against the expanded graph. The converter pins no vocabulary version, so this relationship is established by the behaviour recorded here rather than by a declaration, and it is evidence for the terms these graphs exercise, not for all 467 declared terms or for every historical VCF version.

Two pinning limitations are worth stating plainly because they affect reproduction. The published container image lags the source release: version-tagged images exist, but the newest is 3.0.2, which predates the corrections, so reproducing the results above requires building the image from the 3.0.3 source rather than pulling it. The toolchain inside that image is also not independently pinned. The counts above are converter output; the hand-authored illustration file in the release is smaller, because it omits the allele resources and per-sample genotype and allele-call resources that the converter materializes.

The recorded validation run accepts 21 fixtures and includes a negative control, alongside a regression suite of 101 tests. The published workflow recomputes the reported counts and checks evidence references against the maintained artifacts, and each result in Section 5.2 has a script that reproduces it from the pinned inputs. Review of coverage judgements and mappings nonetheless remains a human task. The established result is full representation of the curated VCF 4.5 inventory with version-aware support across VCF 4.1–4.5, demonstrated to be reproducible with an independent converter on the subset tested — not complete conformance.

5.5. One integration example, with checked answers

The claims above concern representation and query behaviour within the VCF world. To test whether the retained context survives contact with an external annotation, we ran one small end-to-end example and checked its answers against expectations authored in advance.

The example is bounded to a single difficulty: allele-dependent values under VCF 4.5 local alleles. LAD is declared `Number=LR`, meaning one value per allele of *the sample's own* local allele subset named by LAA, not per entry of the record's ALT column. Answering “how deep is this alteration in this sample?” therefore requires the declaration, the sample's LAA and the allele's ordinal simultaneously. The input is a pinned four-site, single-sample VCF 4.5 fixture, each site written twice, once with local alleles and once globally; the annotations are a local, explicitly synthetic snapshot of five flagged alterations, labelled as such in the file, and local rather than federated so the example returns the same answer on every run. The link is made in the query rather than by pre-minted identifiers: an annotation names an alteration by contig, position, reference and alternate allele, and the query joins on those four literals. Three assumptions are recorded because they are what would break the link — a shared assembly, as-written allele identity with no normalization or VRS matching, and that a VCF sample column is a sample and not a patient.

Asking which flagged alterations have supporting read depth of at least 20 returns two rows, identical to the expected answers: `chr1:1 G>C` at depth 30 and `chr1:2 A>T` at depth 25, each traced to its file, record, sample and the FORMAT declaration that gives the field its cardinality. The three flagged alterations absent from the answer are each absent for a stated reason rather than by oversight.

The example is worth reporting because of what it does under a wrong implementation. Run against converter output from before the local-allele correction, the same query and the same annotations return *two rows again* — but `chr1:1 G>A` at 30 and `chr1:2 A>C` at 25, the two alterations a positional reading of `Number=LR` would wrongly report, at exactly the depths predicted in advance. A check comparing result counts passes both graphs. This is the concrete form of the paper's argument: retaining the declaration, the local allele set and the allele ordinal is not bookkeeping, because dropping it yields a plausible, well-formed, wrong answer delivered with complete provenance.

What this example demonstrates is that the representation carries the interpretation context, that a mapping rule connects it to external annotation, and that the answers match those authored beforehand. What it does not demonstrate is equally clear: the input is synthetic and tiny, the annotations are invented and carry no biological or clinical meaning, the link assumes as-written allele identity, and a single fixture establishes correct local-allele handling here rather than in general.

6. Discussion

6.1. Connecting source observations to biomedical knowledge

VCF Core can make more of the evidence already present in a VCF file available to semantic applications. It retains observations and their interpretation context, while other models describe specimens, patients, diseases, treatments and biological variation. These layers can develop independently when their relationships are explicit.

Two laboratories reporting the same alteration in separate files could, after reference-aware normalization, link to a common variation description while each retains its own quality values, sample identities and provenance — so a disease association attaches to the description once rather than being copied into every source record, and a revised interpretation can be retrieved alongside the original observations. PROV, FALDO and the published alignments are starting points; the normalization and instance mappings themselves remain to be implemented and evaluated.

Sample identity needs the same separation. In a tumour/normal analysis two columns may be different specimens from one patient, and treating column names as patient identifiers destroys that distinction. An integration layer can link each `VCFSample` to its specimen and both specimens to the patient through a biomedical model such as HERO-Genomics [2]; VCF Core preserves the source side without replacing the clinical one.

6.2. A shared target for conversion engines

A reusable vocabulary also separates conversion strategy from representation decisions. A clinical site converting one sample for inspection and a research group converting a cohort can choose different sample profiles — expanded values for direct annotation, condensed vectors with selective expansion for scale — while retaining the same file, header and record model rather than growing incompatible vocabularies per workload. A pipeline spanning VCF 4.2 and 4.5 data can likewise select the registry and validation overlay from `fileFormat` instead of silently imposing the newest cardinality on an older declaration.

That suggests an evaluation order for future work: compare recovered source values and query answers first, as Section 5.3 does, and only then measure conversion time, peak memory, serialized size and loading cost. Section 3.3 is explicit that the profiles trade query granularity rather than demonstrated storage savings, and the present results do not establish which implementation is most efficient.

6.3. SPARQL questions that combine genomic and clinical evidence

Section 5.5 established the pattern at the smallest scale that requires it: an external annotation joined to sample-level evidence, with the answer depending on interpretation context that the representation retains. The questions below extend that pattern to clinical settings. They are proposed research scenarios and *not* evaluated workflows: unlike the example in Section 5.5, none has been executed, and each requires additional biomedical data, reviewed instance mappings and appropriate access controls. SPARQL provides the query structure; it does not supply missing interpretation or establish that a returned result is clinically actionable.

Three questions. A *pharmacogenomic* review asks which patients with an active clopidogrel prescription have an externally established CYP2C19 phenotype relevant to a guideline for their indication, and which source observations support that assessment; the CPIC guideline shows why phenotype and indication both matter [1], and a workflow such as PharmCAT separates variant processing, allele matching and clinical interpretation [21] — steps that must not collapse into reading one genotype token as a metabolizer phenotype. A *rare-disease* investigation asks which candidate alterations lie in genes associated with diseases sharing the patient's recorded phenotypic features, joining specimen-linked calls to normalized alterations, gene associations and phenotype descriptions such as HPO [22]; its governing constraint is that a row absent from a variant-only VCF is not evidence that a parent is

homozygous reference. An *oncology* review asks which alterations detected in a tumour have curated therapeutic evidence for the patient's cancer type, joining a resource such as CIViC [23] to normalized alterations and retrieving tumour and normal calls through their specimen links; a missing normal call likewise remains missing evidence, not proof of tumour specificity.

Each reaches its sample-level evidence through the same declaration-to-value path that Section 5.5 executes, and each needs clinical assertions from an application schema that is not part of VCF Core. The vocabulary retains the observations; completeness, segregation and therapeutic interpretation require evidence it does not carry.

These scenarios could be implemented over a materialized graph or, where source interfaces and authorization permit, through federated SPARQL as demonstrated in related work [3]. Availability of RDF and vocabulary alignments alone does not guarantee complete or performant answers. Future experiments should evaluate answer correctness, instance-link accuracy, provenance recovery and source availability before making claims about medical utility or query speed. The opportunity is to investigate these questions against a common representation instead of repeatedly reconstructing VCF semantics within each application.

7. Conclusion

VCF Core provides a shared semantic foundation for representing VCF observations together with the context needed to interpret them. It combines reuse of existing models, explicit source structure, alternative sample representations and version-specific artifacts for VCF 4.1–4.5. All 104 constructs in the curated VCF 4.5 inventory are represented, and the claims that matter have been executed rather than asserted: the specification-derived checks agree across a second producer on 536 of 572 replayed checks with no divergence traceable to the vocabulary, all 35 records round-trip through structured properties, and one integration example returns answers authored in advance together with their source provenance.

The remaining work is specific rather than general. Evidence is absent for 41 of the 94 requirements, concentrated in structural-variant fields, and thinner than we would like for allele-dependent values in particular; three structured accessors are not yet implemented in the cited converter; one QUAL datatype question is an interpretation the vocabulary deliberately leaves open; and the per-sample condensed profile cannot yet answer the hardest missingness question. None of these is a representation gap, and none is hypothetical — each is a named requirement with a recorded result.

What this foundation makes practical is the step from representing genomic files to connecting their contents with biomedical knowledge. Conversion engines can target the same model, integration layers can preserve the distinction between observations and interpretations, and SPARQL studies can examine medically relevant questions with traceable source evidence. The clinical scenarios in Section 6.3 remain future work: the VCF-side representation they would build on is in place and checked, while the biomedical models and reviewed instance mappings they equally require are not.

Acknowledgments

Project funding provided from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N). Ruben Taelman is a postdoctoral fellow of the Research Foundation – Flanders (FWO) (1202124N).

Declaration on Generative AI

OpenAI Codex and Claude assisted with preparatory work on the vocabulary, assessment artifacts and manuscript. ChatGPT assisted with restructuring, prose revision, reference checks and prospective query examples.

References

- [1] C. R. Lee, J. A. Luzum, K. Sangkuhl, R. S. Gammal, M. S. Sabatine, C. M. Stein, D. F. Kisor, N. A. Limdi, Y. M. Lee, S. A. Scott, J.-S. Hulot, D. M. Roden, A. Gaedigk, K. E. Caudle, T. E. Klein, J. A. Johnson, A. R. Shuldiner, Clinical pharmacogenetics implementation consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update, *Clinical Pharmacology & Therapeutics* 112 (2022) 959–967. doi:10.1002/cpt.2526.
- [2] M. Cazzaro, I. G. Gut, L. Menotti, M. Rueda, G. Silvello, HERO-Genomics: An ontology for integration and access of multicenter genomic data, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 33–42. URL: https://ceur-ws.org/Vol-4196/paper_17.pdf.
- [3] A. Bodrug-Schepers, H. Chabane, G. Montoya, P. Serrano-Alvarado, R. Redon, A. Gaignard, Semantic beacons: a framework to support federated querying over genomic variants and public knowledge graphs, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 21–32. URL: https://ceur-ws.org/Vol-4196/paper_16.pdf.
- [4] P. Danecek, A. Auton, G. Abecasis, C. A. Albers, E. Banks, M. A. DePristo, R. E. Handsaker, G. Lunter, G. T. Marth, S. T. Sherry, G. McVean, R. Durbin, 1000 Genomes Project Analysis Group, The variant call format and VCFtools, *Bioinformatics* 27 (2011) 2156–2158. URL: <https://academic.oup.com/bioinformatics/article/27/15/2156/402296>. doi:10.1093/bioinformatics/btr330.
- [5] VCF, The Variant Call Format Specification: VCFv4.5 and BCFv2.2, 2026. URL: <https://samtools.github.io/hts-specs/VCFv4.5.pdf>, 25 February 2026 printing, version e821e4f; accessed 6 September 2026.
- [6] J. Baran, B. S. B. Durgahee, K. Eilbeck, E. Antezana, R. Hoehndorf, M. Dumontier, GFVO: the Genomic Feature and Variation Ontology, *PeerJ* 3 (2015) e933. URL: <https://peerj.com/articles/933/>. doi:10.7717/peerj.933.
- [7] E. D. S. Penha, E. Iriabho, A. Dussaq, D. M. de Oliveira, J. S. Almeida, Isomorphic semantic mapping of variant call format (VCF2RDF), *Bioinformatics* 33 (2017) 547–548. URL: <https://academic.oup.com/bioinformatics/article/33/4/547/2593587>. doi:10.1093/bioinformatics/btw652.
- [8] E. Crum, VCF Core Vocabulary, Version 2.1.2. GitHub repository, 2026. URL: <https://github.com/ecrum19/vcf-core-vocabulary/tree/3f06d18>, version 2.1.2 with the SSSOM alignment set; source snapshot of 14 September 2026.
- [9] E. Crum, R. Taelman, B. Buelens, G. Ertaylan, R. Verborgh, Semantifying genomic variant data: VCF to RDF conversion framework, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 199–200. URL: https://ceur-ws.org/Vol-4196/paper_6.pdf.
- [10] SAM/BAM Format Specification Working Group, HTS format specifications: VCF specifications and earlier versions, Official specification index, 2026. URL: <https://samtools.github.io/hts-specs/>, accessed 9 September 2026.
- [11] World Wide Web Consortium, PROV-O: The PROV Ontology, W3C Recommendation, World Wide Web Consortium, 2013. URL: <https://www.w3.org/TR/2013/REC-prov-o-20130430/>.
- [12] K. Eilbeck, S. E. Lewis, C. J. Mungall, M. Yandell, L. Stein, R. Durbin, M. Ashburner, The sequence ontology: a tool for the unification of genome annotations, *Genome Biology* 6 (2005) R44. URL: <https://link.springer.com/article/10.1186/gb-2005-6-5-r44>. doi:10.1186/gb-2005-6-5-r44.
- [13] J. T. Bolleman, C. J. Mungall, F. Strozzi, J. Baran, M. Dumontier, R. J. P. Bonnal, R. Buels, R. Hoehndorf, T. Fujisawa, T. Katayama, P. J. A. Cock, FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation, *Journal of Biomedical Semantics* 7 (2016) 39. URL: <https://link.springer.com/article/10.1186/s13326-016-0067-z>. doi:10.1186/s13326-016-0067-z.

- [14] S. Kawashima, T. Fujisawa, T. Katayama, Genome variation ontology for annotation of complex structural variations, in: *Proceedings of the 14th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2023)*, volume 3415 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 126–127. URL: <https://ceur-ws.org/Vol-3415/paper-22.pdf>.
- [15] A. H. Wagner, L. Babb, G. Alterovitz, M. Baudis, M. Brush, D. L. Cameron, M. Cline, M. Griffith, O. L. Griffith, S. E. Hunt, D. Kreda, J. M. Lee, S. Li, J. Lopez, E. Moyer, T. Nelson, R. Y. Patel, K. Riehle, P. N. Robinson, S. Rynearson, H. Schuilenburg, K. Tsukanov, B. Walsh, M. Konopko, H. L. Rehm, A. D. Yates, R. R. Freimuth, R. K. Hart, The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification, *Cell Genomics* 1 (2021) 100027. doi:10.1016/j.xgen.2021.100027.
- [16] N. Matentzoglou, J. P. Balhoff, S. M. Bello, C. Bizon, M. Brush, T. J. Callahan, C. G. Chute, W. D. Duncan, C. T. Evelo, D. Gabriel, J. Graybeal, A. Gray, B. M. Gyori, M. Haendel, H. Harmse, N. L. Harris, I. Harrow, H. B. Hegde, A. L. Hoyt, C. T. Hoyt, D. Jiao, E. Jiménez-Ruiz, S. Jupp, H. Kim, S. Koehler, T. Liener, Q. Long, J. Malone, J. A. McLaughlin, J. A. McMurry, S. Moxon, M. C. Munoz-Torres, D. Osumi-Sutherland, J. A. Overton, B. Peters, T. Putman, N. Queralt-Rosinach, K. Shefchek, H. Solbrig, A. Thessen, T. Tudorache, N. Vasilevsky, A. H. Wagner, C. J. Mungall, A simple standard for sharing ontological mappings (SSSOM), *Database* 2022 (2022) baac035. URL: <https://academic.oup.com/database/article/doi/10.1093/database/baac035/6591806>. doi:10.1093/database/baac035.
- [17] World Wide Web Consortium, SKOS Simple Knowledge Organization System Reference, W3C Recommendation, World Wide Web Consortium, 2009. URL: <https://www.w3.org/TR/skos-reference/>.
- [18] SWAT4HCLS 2025 BioHackathon genomic variation participants, Genomic variant RDF representations: mapping declarations to GA4GH VRS, Project site and report, 2025. URL: <https://swat4hcls-2025-genomic-variation.github.io/genomic-variant-schema/>, declarations transcribed as SSSOM sets and republished with the VCF Core Vocabulary; accessed 8 September 2026.
- [19] World Wide Web Consortium, Shapes Constraint Language (SHACL), W3C Recommendation, World Wide Web Consortium, 2017. URL: <https://www.w3.org/TR/2017/REC-shacl-20170720/>.
- [20] VCF-RDFizer maintainers, VCF-RDFizer, Version 3.0.3. Open-source software, 2026. URL: <https://github.com/ecrum19/VCF-RDFizer/tree/be658a2>, tag v3.0.3; converted and validated 14 September 2026.
- [21] PharmCAT contributors, PharmCAT methods, Official documentation, 2026. URL: <https://pharmcat.org/methods/>, accessed 9 September 2026.
- [22] Human Phenotype Ontology Consortium, The human phenotype ontology, Project website and documentation, 2026. URL: <https://hpo.jax.org/>, accessed 9 September 2026.
- [23] M. Griffith, N. C. Spies, K. Krysiak, J. F. McMichael, A. C. Coffman, A. M. Danos, B. J. Ainscough, C. A. Ramirez, D. T. Rieke, L. Kujan, E. K. Barnell, A. H. Wagner, Z. L. Skidmore, A. Wollam, C. J. Liu, M. R. Jones, R. L. Bilski, R. Lesurf, Y.-Y. Feng, N. M. Shah, M. Bonakdar, L. Trani, M. Matlock, A. Ramu, K. M. Campbell, G. C. Spies, A. P. Graubert, K. Gangavarapu, J. M. Eldred, D. E. Larson, J. R. Walker, B. M. Good, C. Wu, A. I. Su, R. Dienstmann, A. A. Margolin, D. Tamborero, N. Lopez-Bigas, S. J. M. Jones, R. Bose, D. H. Spencer, L. D. Wartman, R. K. Wilson, E. R. Mardis, O. L. Griffith, CIViC is a community knowledgebase for expert crowdsourcing the clinical interpretation of variants in cancer, *Nature Genetics* 49 (2017) 170–174. doi:10.1038/ng.3774.